САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №4
по курсу «Проектирование и реализация баз данных»
Вариант 10 “Автовокзал”

Выполнил:
Гашимов И.Ф.
К3239
Проверила:
Говорова М.М.

Санкт-Петербург

2026 г.

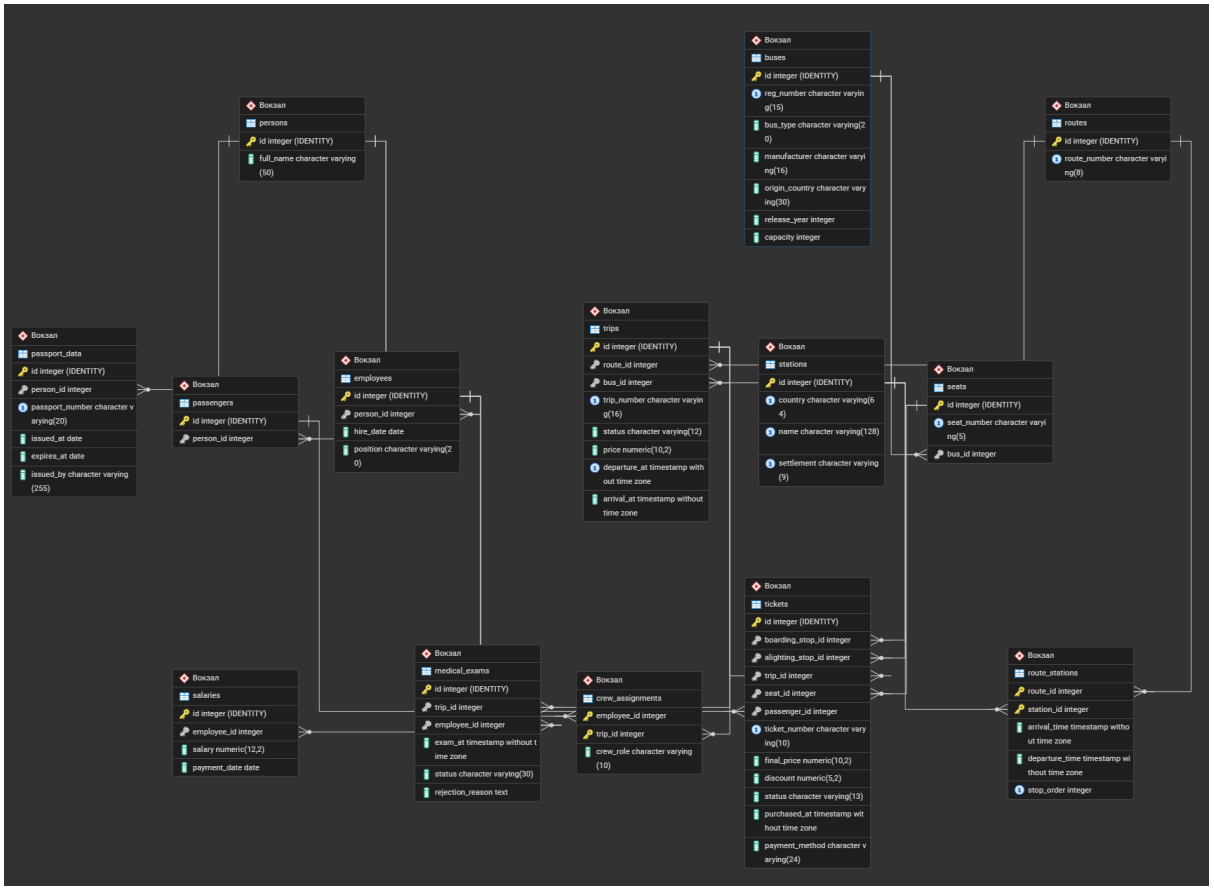
Цель работы: овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание:

- создать запросы и представления на выборку данных согласно индивидуальному заданию;
- составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с подзапросами;
- изучить графическое представление запросов и историю запросов;
- создать простой и составной индексы, сравнить время выполнения запросов без индексов и с индексами, использовать EXPLAIN.

Схема базы данных

В БД созданы таблицы: Маршрут, Остановка, Остановка_в_маршруте, Рейс, Билет, Автобус, Место, Пассажир, Медосмотр, Сотрудник, Зарплата, Физ_лицо, Паспорт_данные, Назначен_в_экипаж.



ERD в нативном качестве есть в репозитории 3 работы

1. Выполнение работы

1.1 Запросы на выборку данных

- Вывести фамилии водителей и номера автобусов, отправившиеся в рейсы до 12 часов текущего дня.
- Рассчитать выручку от продажи билетов за прошедший день.
- Вывести список водителей, которые не выполнили ни одного рейса за прошедшую неделю.
- Вывести сумму убытков из-за непроданных мест в автобусе за прошедшую неделю.
- Найт самый популярный маршрут за прошедший месяц.
- Вывести тип автобуса, который используется на всех рейсах.
- Вывести данные водителя, который провел максимальное время в пути за прошедшую неделю.

1 Запрос: Вывести фамилии водителей и номера автобусов, отправившиеся в рейсы до 12 часов текущего дня.

```
SELECT DISTINCT
   fl."фио_лица" AS "Водитель",
   a."номер" AS "Номер_автобуса",
   r."номер" AS "Номер_рейса",
   r."дата_отправления" AS "Отправление"
FROM "Avtovokzal"."Рейс" r
JOIN "Avtovokzal"."Автобус" a
   ON a."ид_автобуса" = r."ид_автобуса"
JOIN "Avtovokzal"."Назначен_в_экипаж" ne
   ON ne."ид_рейса" = r."ид_рейса"
JOIN "Avtovokzal"."Сотрудник" s
   ON s."ид_сотрудника" = ne."ид_сотрудника"
JOIN "Avtovokzal"."Физ_лицо" fl
   ON fl."ид_лица" = s."ид_лица"
WHERE s."должность" = 'Водитель'
  AND r."статус" IN ('Отправлен', 'Прибыл')
  AND r."дата_отправления" >= CURRENT_DATE
  AND r."дата_отправления" < CURRENT_DATE + INTERVAL '12 hours'
ORDER BY r."дата_отправления", fl."фио_лица";
```

Запрос выбирает рейсы, которые отправились сегодня до 12:00. Далее через таблицу назначения в экипаж определяется, какие сотрудники были назначены на рейс, и оставляются только сотрудники с должностью «Водитель». В результате выводятся ФИО водителя, номер автобуса, номер рейса и время отправления.

2 Запрос: Рассчитать выручку от продажи билетов за прошедший день.

```
SELECT
   COALESCE(SUM(b."цена"), 0) AS "Выручка_за_вчера"
FROM "Avtovokzal"."Билет" b
WHERE b."дата_покупки" >= CURRENT_DATE - INTERVAL '1 day'
  AND b."дата_покупки" < CURRENT_DATE
  AND b."статус" IN ('Куплен', 'Использован');
```

Запрос суммирует стоимость всех билетов, купленных за предыдущий день. Учитываются только билеты со статусами «Куплен» и «Использован», чтобы исключить отмененные или недействительные продажи.

3 Запрос: Вывести список водителей, которые не выполнили ни одного рейса за прошедшую неделю.

```
SELECT
   fl."фио_лица" AS "Водитель"
FROM "Avtovokzal"."Сотрудник" s
JOIN "Avtovokzal"."Физ_лицо" fl
   ON fl."ид_лица" = s."ид_лица"
WHERE s."должность" = 'Водитель'
  AND NOT EXISTS (
     SELECT 1
     FROM "Avtovokzal"."Назначен_в_экипаж" ne
     JOIN "Avtovokzal"."Рейс" r
        ON r."ид_рейса" = ne."ид_рейса"
     WHERE ne."ид_сотрудника" = s."ид_сотрудника"
       AND r."статус" IN ('Отправлен', 'Прибыл')
       AND r."дата_отправления" >= CURRENT_DATE - INTERVAL '7 days'
       AND r."дата_отправления" < CURRENT_DATE
  )
ORDER BY fl."фио_лица";
```

Запрос находит всех сотрудников с должностью «Водитель» и проверяет, были ли у них рейсы за последние 7 дней. Если у водителя не найдено ни одного отправленного или завершенного рейса, он попадает в результат.

4 Запрос: Вывести сумму убытков из-за непроданных мест в автобусе за прошедшую неделю.

```
WITH sold AS (
   SELECT
      b."ид_рейса",
      COUNT(*) AS sold_seats
   FROM "Avtovokzal"."Билет" b
   WHERE b."ид_места" IS NOT NULL
```

```
    AND b."статус" IN ('Куплен', 'Использован')
  GROUP BY b."ид_рейса"
)
SELECT
  COALESCE(
    SUM(
      GREATEST(a."количество_мест" - COALESCE(s.sold_seats, 0), 0)
      * COALESCE(r."цена", 0)
    ),
    0
  )::numeric(12,2) AS "Убытки_из_за_непроданных_мест"
FROM "Avtovokzal"."Рейс" r
JOIN "Avtovokzal"."Автобус" a
  ON a."ид_автобуса" = r."ид_автобуса"
LEFT JOIN sold s
  ON s."ид_рейса" = r."ид_рейса"
WHERE r."статус" IN ('Отправлен', 'Прибыл')
  AND r."дата_отправления" >= CURRENT_DATE - INTERVAL '7 days'
  AND r."дата_отправления" < CURRENT_DATE;
```

Сначала подсчитывается количество проданных мест по каждому рейсу. Затем для каждого рейса определяется число непроданных мест как разность между вместимостью автобуса и числом проданных билетов. После этого количество непроданных мест умножается на цену рейса, и полученные значения суммируются за последнюю неделю.

5 Запрос: Найти самый популярный маршрут за прошедший месяц.

```
SELECT
  m."номер_маршрута" AS "Номер_маршрута",
  COUNT(r."ид_рейса") AS "Количество_рейсов"
FROM "Avtovokzal"."Рейс" r
JOIN "Avtovokzal"."Маршрут" m
  ON m."ид_маршрута" = r."ид_маршрута"
WHERE r."статус" IN ('Отправлен', 'Прибыл')
  AND r."дата_отправления" >= CURRENT_DATE - INTERVAL '1 month'
  AND r."дата_отправления" < CURRENT_DATE
GROUP BY m."ид_маршрута", m."номер_маршрута"
ORDER BY COUNT(r."ид_рейса") DESC, m."номер_маршрута"
LIMIT 1;
```

Запрос подсчитывает количество выполненных рейсов по каждому маршруту за последний месяц. Для этого таблица Рейс соединяется с таблицей Маршрут, после чего данные группируются по номеру маршрута. В результат выводится маршрут, по которому было выполнено наибольшее количество рейсов.

6 Запрос: Вывести тип автобуса, который используется на всех рейсах. (Возможно имелось ввиду Тип автобуса, который используется на всех маршрутах. То есть для каждого маршрута есть хотя бы один рейс этого типа.)

```
SELECT
a."тип" AS "Тип_автобуса"
FROM "Avtovokzal"."Рейс" r
JOIN "Avtovokzal"."Автобус" a
ON a."ид_автобуса" = r."ид_автобуса"
GROUP BY a."тип"
HAVING COUNT(DISTINCT r."ид_маршрута") = (
SELECT COUNT(*)
FROM "Avtovokzal"."Маршрут"
)
ORDER BY a."тип";
```

Запрос определяет, на скольких различных маршрутах использовался каждый тип автобуса. Если количество маршрутов для данного типа совпадает с общим количеством маршрутов в базе данных, значит этот тип автобуса используется на всех маршрутах.

7 Запрос: Вывести данные водителя, который провел максимальное время в пути за прошедшую неделю.

```
WITH driver_time AS (
   SELECT
      s."ид_сотрудника",
      fl."фио_лица",
      SUM(r."дата_прибытия" - r."дата_отправления") AS "общее_время_в_пути"
   FROM "Avtovokzal"."Назначен_в_экипаж" ne
   JOIN "Avtovokzal"."Сотрудник" s
      ON s."ид_сотрудника" = ne."ид_сотрудника"
   JOIN "Avtovokzal"."Физ_лицо" fl
      ON fl."ид_лица" = s."ид_лица"
   JOIN "Avtovokzal"."Рейс" r
      ON r."ид_рейса" = ne."ид_рейса"
   WHERE s."должность" = 'Водитель'
     AND r."статус" IN ('Отправлен', 'Прибыл')
     AND r."дата_отправления" >= CURRENT_DATE - INTERVAL '7 days'
     AND r."дата_отправления" < CURRENT_DATE
   GROUP BY s."ид_сотрудника", fl."фио_лица"
)
SELECT
   dt."фио_лица" AS "Водитель",
   pd."номер" AS "Паспорт",
   dt."общее_время_в_пути"
FROM driver_time dt
```

```
JOIN "Avtovokzal"."Сотрудник" s
   ON s."ид_сотрудника" = dt."ид_сотрудника"
LEFT JOIN "Avtovokzal"."Паспорт_данные" pd
   ON pd."ид_лица" = s."ид_лица"
WHERE dt."общее_время_в_пути" = (
   SELECT MAX("общее_время_в_пути")
   FROM driver_time
);
```

Представление показывает рейсы на завтра и количество свободных мест по каждому из них. Количество занятых мест определяется по числу проданных билетов с указанным местом, а количество свободных мест рассчитывается как разность между вместимостью автобуса и числом занятых мест.

1.2. Создание представлений

1 Представление: Количество свободных мест на все рейсы на завтра.

```
CREATE OR REPLACE VIEW "Avtovokzal"."vw_свободные_места_на_завтра" AS
SELECT
   r."ид_рейса",
   r."номер" AS "Номер_рейса",
   m."номер_маршрута" AS "Номер_маршрута",
   r."дата_отправления" AS "Отправление",
   a."номер" AS "Номер_автобуса",
   a."тип" AS "Тип_автобуса",
   a."количество_мест" AS "Всего_мест",
   COUNT(b."ид_билета") AS "Занято_мест",
   a."количество_мест" - COUNT(b."ид_билета") AS "Свободно_мест"
FROM "Avtovokzal"."Рейс" r
JOIN "Avtovokzal"."Автобус" a
   ON a."ид_автобуса" = r."ид_автобуса"
JOIN "Avtovokzal"."Маршрут" m
   ON m."ид_маршрута" = r."ид_маршрута"
LEFT JOIN "Avtovokzal"."Билет" b
   ON b."ид_рейса" = r."ид_рейса"
  AND b."ид_места" IS NOT NULL
  AND b."статус" IN ('Куплен', 'Использован')
WHERE r."дата_отправления" >= CURRENT_DATE + INTERVAL '1 day'
  AND r."дата_отправления" < CURRENT_DATE + INTERVAL '2 days'
  AND r."статус" <> 'Отменён'
GROUP BY
   r."ид_рейса",
   r."номер",
   m."номер_маршрута",
   r."дата_отправления",
   a."номер",
   a."тип",
```

```
  a."количество_мест";
```

Видим такой вывод:

Показаны строки: от 1 до 80 Номер страницы: 1 из 1

	ид_рейса integer	Номер_рейса character varying (16)	Номер_маршрута character varying (8)	Отправление timestamp without time zone	Номер_автобуса character varying (15)	Тип_автобуса character varying (20)	Всего_мест integer	Занято_мест bigint	Свободно_мест bigint
1	53	REIS-1053	936B	2025-12-11 15:45:00	ZA195BB47	Туристический	39	21	18
2	48	REIS-1048	940A	2025-12-12 12:00:00	SJ187BB41	Междугородний	55	13	42
3	43	REIS-1043	263M	2025-12-19 12:15:00	SJ187BB41	Междугородний	55	23	32
4	49	REIS-1049	979B	2025-12-22 17:30:00	HL283BB48	Междугородний	31	18	13
5	54	REIS-1054	371	2025-12-31 13:15:00	DC510BB72	Междугородний	55	20	35
6	47	REIS-1047	371	2025-12-31 16:30:00	HX516BB72	Туристический	44	11	33
7	44	REIS-1044	993	2026-01-03 11:45:00	HL283BB48	Междугородний	31	16	15

Представление показывает рейсы на завтра и количество свободных мест по каждому из них. Свободные места определяются как разность между вместимостью автобуса и числом уже занятых мест по проданным билетам.

2 Представление: Самый популярный маршрут этой зимой.

```
CREATE OR REPLACE VIEW
"Avtovokzal"."vw_самый_популярный_маршрут_этой_зимой" AS
SELECT
   m."номер_маршрута" AS "Номер_маршрута",
   COUNT(r."ид_рейса") AS "Количество_рейсов"
FROM "Avtovokzal"."Рейс" r
JOIN "Avtovokzal"."Маршрут" m
   ON m."ид_маршрута" = r."ид_маршрута"
WHERE r."статус" IN ('Отправлен', 'Прибыл')
  AND EXTRACT(MONTH FROM r."дата_отправления") IN (12, 1, 2)
GROUP BY m."номер_маршрута"
ORDER BY COUNT(r."ид_рейса") DESC;
```

Представление определяет количество выполненных рейсов по каждому маршруту в зимний период (декабрь, январь и февраль). Популярность маршрута оценивается по числу рейсов. Результат упорядочен по убыванию количества рейсов, поэтому первый маршрут в списке является самым популярным.

Видим такой вывод:

Результат Сообщения Уведомления

Показаны строки: от 1 до 8

	Номер_маршрута character varying (8)	Количество_рейсов bigint
1	371	3
2	263M	2
3	940A	2
4	993	2
5	936B	1
6	979B	1
7	508Б	1
8	775A	1

1.3 Запросы на модификацию данных

Запрос INSERT с подзапросом:

```
INSERT INTO "Avtovokzal"."Зарплата" ("ид_сотрудника", "оклад", "дата_выплаты")
SELECT
   s."ид_сотрудника",
   COALESCE(
      (
         SELECT z."оклад"
         FROM "Avtovokzal"."Зарплата" z
         WHERE z."ид_сотрудника" = s."ид_сотрудника"
         ORDER BY z."дата_выплаты" DESC
         LIMIT 1
      ),
      60000
   ) AS "оклад",
   CURRENT_DATE
FROM "Avtovokzal"."Сотрудник" s
WHERE s."ид_сотрудника" = (
   SELECT ne."ид_сотрудника"
   FROM "Avtovokzal"."Назначен_в_экипаж" ne
   GROUP BY ne."ид_сотрудника"
   ORDER BY COUNT(*) DESC, ne."ид_сотрудника"
   LIMIT 1
);
```

Запрос добавляет новую запись о зарплате сотруднику, который чаще всего участвовал в рейсах. Размер оклада берется из его последней записи о зарплате, а если таких записей нет, подставляется значение 60000.

Ниже видим что запись действительно появилась:

Результат Сообщения Уведомления

Показаны строки: от 1 до 3 Номер страницы: 1 из 1

	ид_зарплаты [PK] integer	ид_сотрудника integer	оклад numeric (12,2)	дата_выплаты date
1	49	4	69170.00	2026-04-12
2	8	4	69170.00	2026-03-13
3	7	4	51315.00	2026-02-11

Запрос UPDATE с подзапросом:

Сначала выберем билет со статусом “Куплен” для последнего завершенного рейса. Покажем запись до изменения:

Показаны строки: от 1 до 1 Номер страницы: 1 из 1

	ид_билета [PK] integer	ид_остановки_посадки integer	ид_остановки_высадки integer	ид_рейса integer	ид_места integer	ид_пассажира integer	номер character varying (10)	цена numeric (10,2)	скидка numeric (5,2)	статус character varying (13)	дата_п timesta
1	655	38	7	35	333	20	T609529	3381.00	0.00	Куплен	2026-0

```
UPDATE "Avtovokzal"."Билет" b
SET "статус" = 'Использован'
WHERE b."ид_билета" = (
   SELECT b2."ид_билета"
   FROM "Avtovokzal"."Билет" b2
   WHERE b2."статус" = 'Куплен'
     AND b2."ид_рейса" = (
        SELECT r."ид_рейса"
        FROM "Avtovokzal"."Рейс" r
        WHERE r."дата_прибытия" < NOW()
          AND r."статус" IN ('Отправлен', 'Прибыл')
        ORDER BY r."дата_прибытия" DESC
        LIMIT 1
     )
   ORDER BY b2."дата_покупки" DESC
   LIMIT 1
);
```

Запрос изменяет статус выбранного билета с «Куплен» на «Использован». Подзапрос определяет нужный билет автоматически.

После выполнения видим:

Результат Сообщения Уведомления

Показаны строки: от 1 до 1 Номер страницы: 1 из 1

	ид_билета [PK] integer	ид_остановки_посадки integer	ид_остановки_высадки integer	ид_рейса integer	ид_места integer	ид_пассажира integer	номер character varying (10)	цена numeric (10,2)	скидка numeric (5,2)	статус character varying (13)	дата_по timestar
1	656	38	16	35	359	51	T397252	3381.00	0.00	Использован	2026-04

Запрос DELETE с подзапросом:

Сначала выбирается последняя запись о зарплате водителя, который участвовал в минимальном количестве рейсов. Видим до удаления:

	ид_зарплаты [PK] integer	ид_сотрудника integer	оклад numeric (12,2)	дата_выплаты date
1	14	7	44255.00	2026-03-13

```
DELETE FROM "Avtovokzal"."Зарплата" z
WHERE z."ид_зарплаты" = (
   SELECT zp."ид_зарплаты"
   FROM "Avtovokzal"."Зарплата" zp
   WHERE zp."ид_сотрудника" = (
      SELECT s."ид_сотрудника"
      FROM "Avtovokzal"."Сотрудник" s
      LEFT JOIN "Avtovokzal"."Назначен_в_экипаж" ne
         ON ne."ид_сотрудника" = s."ид_сотрудника"
      WHERE s."должность" = 'Водитель'
      GROUP BY s."ид_сотрудника"
      ORDER BY COUNT(ne."ид_рейса") ASC, s."ид_сотрудника"
      LIMIT 1
   )
   ORDER BY zp."дата_выплаты" DESC, zp."ид_зарплаты" DESC
   LIMIT 1
);
```

Запрос удаляет последнюю запись о зарплате у водителя с минимальным числом рейсов. Нужная запись выбирается через подзапрос.

1.4 Работа с индексами и планами запросов

1.4.1 Простой индекс

вводим этот запрос без индекса:

```
SELECT
   COALESCE(SUM(b."цена"), 0) AS "Выручка_за_вчера"
FROM "Avtovokzal"."Билет" b
WHERE b."дата_покупки" >= CURRENT_DATE - INTERVAL '1 day'
  AND b."дата_покупки" < CURRENT_DATE
  AND b."статус" IN ('Куплен', 'Использован');
```

Explain:

Explain Analyze:

Graphical | Analysis | Statistics

#	Node	Строки Actual	Loops
1.	→ Aggregate (rows=1 loops=1)	1	1
2.	→ Seq Scan on Билет as b (rows=35 loops=1) **Фильтр**: ((("статус")::text = ANY ('{Куплен,Использован}'::text[])) AND ("дата_покупки" < CURRENT_DATE) AND ("дата_по **Строки, удаленные фильтром**: 1449	35	1

Видим время выполнения:

Результат | Сообщения | План выполнения × | Уведомления

```
Запрос выполнен успешно. Общее время выполнения: 53 мс.
обработано строк: 1.
```

Создадим простой индекс

```
CREATE INDEX "idx_билет_дата_покупки"
ON "Avtovokzal"."Билет" ("дата_покупки");
```

Результат | Сообщения | План выполнения × | Уведомления

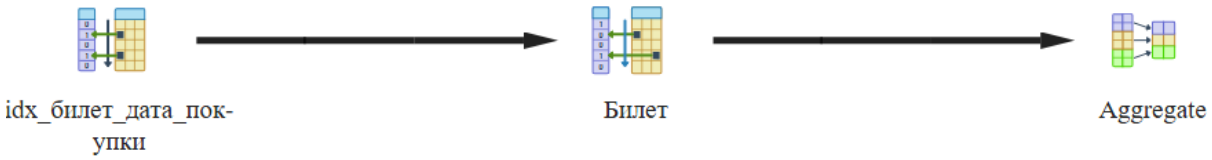
```
CREATE INDEX

Запрос завершён успешно, время выполнения: 53 мс.
```

Создан простой индекс по полю дата_покупки, так как в запросе отбор записей выполняется именно по этой дате.

Выполняем повторно запрос с индексом:

```
SELECT
   COALESCE(SUM(b."цена"), 0) AS "Выручка_за_вчера"
FROM "Avtovokzal"."Билет" b
WHERE b."дата_покупки" >= CURRENT_DATE - INTERVAL '1 day'
  AND b."дата_покупки" < CURRENT_DATE
  AND b."статус" IN ('Куплен', 'Использован');
```

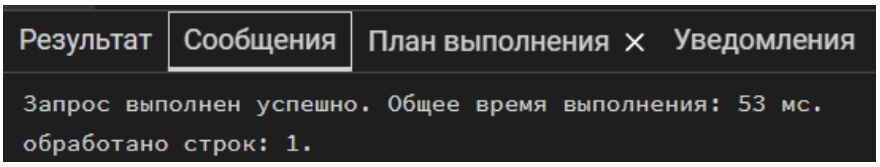
Explain:



Explain Analyze:

#	Node	Строки Actual	Loops
1.	→ Aggregate (rows=1 loops=1)	1	1
2.	→ Bitmap Heap Scan on Билет as b (rows=35 loops=1) **Фильтр**: (("статус")::text = ANY ('{Куплен,Использован}'::text[])) **Строки, удаленные фильтром**: 2 **Перепровеить условие**: (("дата_покупки" >= (CURRENT_DATE - '1 day'::interval)) AND ("дата_покупки" < CURRENT_DATE **Блоки кучи**: exact=5	35	1
3.	→ Bitmap Index Scan using idx_билет_дата_покупки (rows=37 loops=1) **Индекс состояния**: (("дата_покупки" >= (CURRENT_DATE - '1 day'::interval)) AND ("дата_покупки" < CURRENT_DATE	37	1

Видим время выполнения:



После создания индекса запрос выполняется повторно для сравнения плана выполнения и времени работы.

Вариант	Время выполнения
Без индекса	53 ms
С индексом	53 ms

В нашем случае индекс играет небольшую роль т.к. у нас не так много строк.

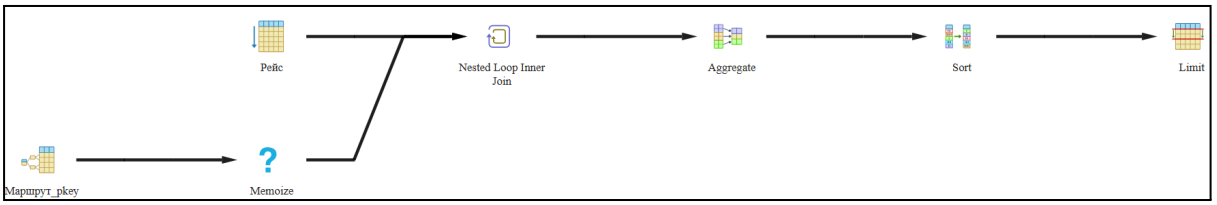
1.4.2 Составной индекс

Теперь делаем то же самое, но для второго запроса.
Возьмём запрос о самом популярном маршруте за прошедший месяц.

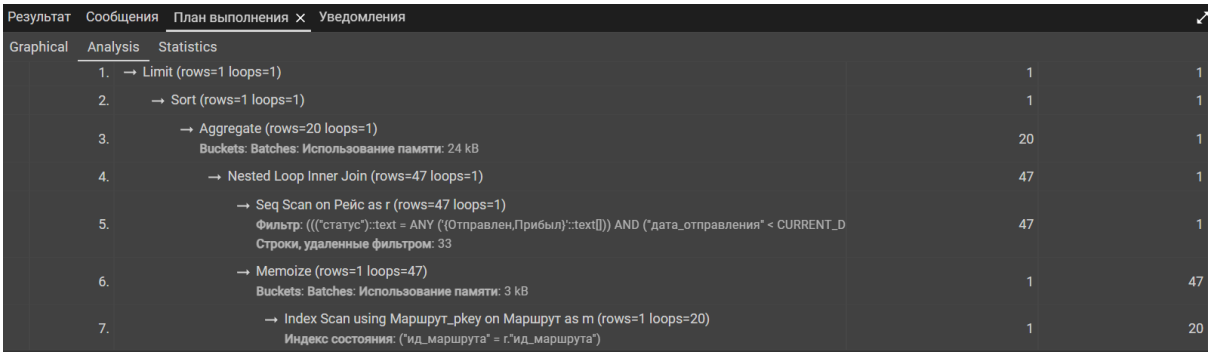
Выполняем запрос без индекса:

```
SELECT
   m."номер_маршрута" AS "Номер_маршрута",
   COUNT(r."ид_рейса") AS "Количество_рейсов"
FROM "Avtovokzal"."Рейс" r
JOIN "Avtovokzal"."Маршрут" m
   ON m."ид_маршрута" = r."ид_маршрута"
WHERE r."статус" IN ('Отправлен', 'Прибыл')
  AND r."дата_отправления" >= CURRENT_DATE - INTERVAL '1 month'
  AND r."дата_отправления" < CURRENT_DATE
GROUP BY m."ид_маршрута", m."номер_маршрута"
ORDER BY COUNT(r."ид_рейса") DESC, m."номер_маршрута"
LIMIT 1;
```

Explain:



Explain Analyze:



Результат Сообщения План выполнения Уведомления

Graphical Analysis Statistics

1.	→ Limit (rows=1 loops=1)	1	1
2.	→ Sort (rows=1 loops=1)	1	1
3.	→ Aggregate (rows=20 loops=1) **Buckets: Batches: Использование памяти**: 24 kB	20	1
4.	→ Nested Loop Inner Join (rows=47 loops=1)	47	1
5.	→ Seq Scan on Рейс as r (rows=47 loops=1) **Фильтр**: ((("статус")::text = ANY ('{Отправлен,Прибыл}'::text[])) AND ("дата_отправления" < CURRENT_D **Строки, удаленные фильтром**: 33	47	1
6.	→ Memoize (rows=1 loops=47) **Buckets: Batches: Использование памяти**: 3 kB	1	47
7.	→ Index Scan using Маршрут_pkey on Маршрут as m (rows=1 loops=20) **Индекс состояния**: ("ид_маршрута" = r."ид_маршрута")	1	20

Время выполнения:

```
Результат  Сообщения  План выполнения ×  Уведомления

Запрос выполнен успешно. Общее время выполнения: 61 мс.
обработано строк: 1.
```

Запрос определяет маршрут, по которому за последний месяц было выполнено максимальное количество рейсов. Сначала выполняется вариант без индекса.

Создадим составной индекс:

```
CREATE INDEX "idx_рейс_дата_маршрут"
ON "Avtovokzal"."Рейс" ("дата_отправления", "ид_маршрута");
```

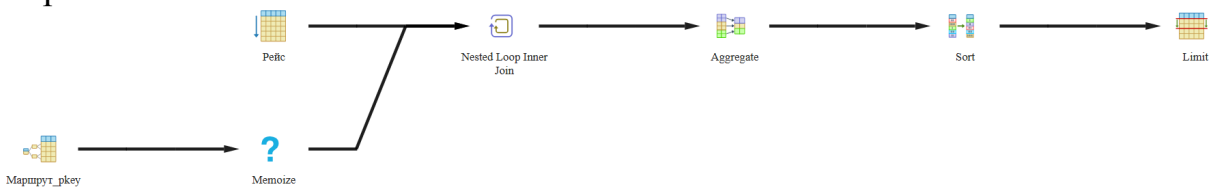
```
Результат  Сообщения  План выполнения ×  Уведомления

CREATE INDEX

Запрос завершён успешно, время выполнения: 53 мс.
```

Создан составной индекс по полям дата_отправления и ид_маршрута, так как в запросе отбор выполняется по дате, а группировка связана с маршрутом.

Выполняем повторно запрос с индексом:

```
SELECT
   m."номер_маршрута" AS "Номер_маршрута",
   COUNT(r."ид_рейса") AS "Количество_рейсов"
FROM "Avtovokzal"."Рейс" r
JOIN "Avtovokzal"."Маршрут" m
   ON m."ид_маршрута" = r."ид_маршрута"
WHERE r."статус" IN ('Отправлен', 'Прибыл')
  AND r."дата_отправления" >= CURRENT_DATE - INTERVAL '1 month'
  AND r."дата_отправления" < CURRENT_DATE
GROUP BY m."ид_маршрута", m."номер_маршрута"
ORDER BY COUNT(r."ид_рейса") DESC, m."номер_маршрута"
LIMIT 1;
```

Explain:



Explain скорее всего не изменился так как сработали оптимизаторы и система решила, что и без этого будет быстро работать, но во времени разница будет.

Explain Analyze:

Graphical Analysis Statistics

1.	→ Limit (rows=1 loops=1)	1	1
2.	→ Sort (rows=1 loops=1)	1	1
3.	→ Aggregate (rows=20 loops=1) **Buckets**: **Batches**: **Использование памяти**: 24 kB	20	1
4.	→ Nested Loop Inner Join (rows=47 loops=1)	47	1
5.	→ Seq Scan on Рейс as r (rows=47 loops=1) **Фильтр**: ((("статус")::text = ANY ('{Отправлен,Прибыл}'::text[])) AND ("дата_отправления" < CURRENT_D **Строки, удаленные фильтром**: 33	47	1
6.	→ Memoize (rows=1 loops=47) **Buckets**: **Batches**: **Использование памяти**: 3 kB	1	47
7.	→ Index Scan using Маршрут_pkey on Маршрут as m (rows=1 loops=20) **Индекс состояния**: ("ид_маршрута" = r."ид_маршрута")	1	20

Время выполнения:

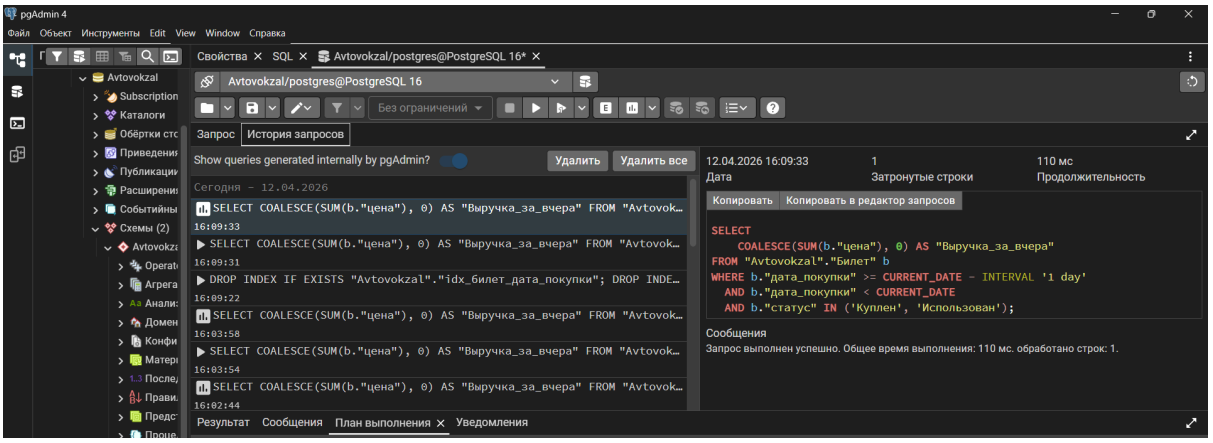
Результат | Сообщения | План выполнения × | Уведомления

```
Запрос выполнен успешно. Общее время выполнения: 52 мс.
обработано строк: 1.
```

Вариант	Время выполнения
Без индекса	61 ms
С индексом	52 ms

1.4.3 История запросов Query History

В процессе выполнения лабораторной работы использовалась вкладка Query History в Query Tool pgAdmin 4. Она позволяет просматривать историю выполненных запросов, включая текст SQL-команды, время выполнения, количество возвращенных строк и служебные сообщения сервера



Выводы

В ходе лабораторной работы были выполнены запросы на выборку данных и созданы представления для базы данных “Автовокзал”. Также были составлены запросы на модификацию данных INSERT, UPDATE и DELETE с использованием подзапросов. Для двух запросов были построены планы выполнения с помощью EXPLAIN, созданы простой и составной индексы и выполнено сравнение времени выполнения запросов до и после индексации. В результате были получены практические навыки работы с Query Tool в pgAdmin 4, анализа планов выполнения запросов и использования индексов для повышения производительности.